Цель работы: изучить функциональные возможности среды разработки Arduino IDE, научиться компилировать программу и загружать её в микроконтроллер. Изучить стандартные библиотеки среды Arduino IDE для работы с портами ввода вывода, интерфейсами микроконтроллера Atmega 328p и научиться применять их на практике.

Немного общих сведений

Выполнение лабораторной работы осуществляется на микроконтроллере Atmega328P, изображённом на рисунке 1.

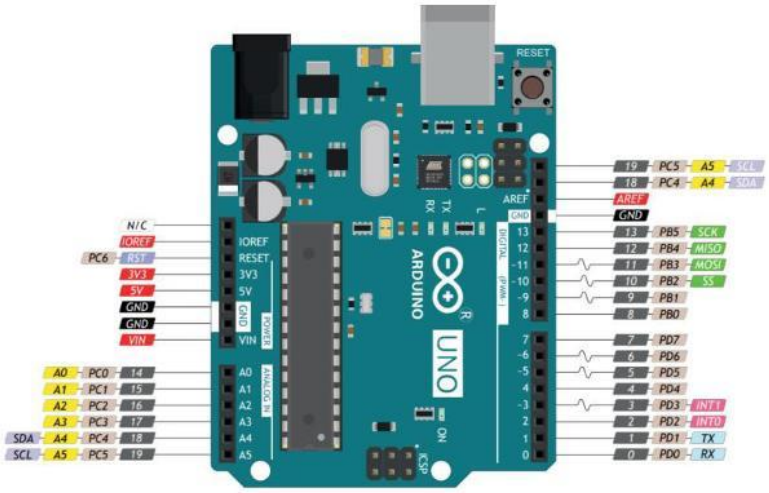


Рисунок 1. Микроконтроллер Atmega 328P на базе Arduino Uno.

Задания для лабораторной работы

1. Получить навыки работы с GPIO портами. Зажечь светодиод, подключенный к порту D13, помигать им.

GPIO порты могут выступать в роли входа и в роли выхода, поэтому перед работой с ними в сегменте инициализации будем устанавливать режим работы порта с помощью команды pinMode(pin, mode), где pin – номер порта, а mode – соответственно, режим работы (INPUT, OUTPUT – используемые режимы).

Порт D13, куда подключен светодиод (рис. 1), в среде Arduino именуется как «LED_BUILTIN». Это имя и будем использовать. Устанавливаем порт в режим OUTPUT. На этом инициализация закончена.

Чтобы зажечь светодиод, нужно подать на него высокий уровень – HIGH, и низкий чтобы погасить – LOW. Ставим задержку в 1000мс между переключением уровня сигнала с одного на другой, таким образом, осуществляя процесс «мигания» светодиода (рис. 2).

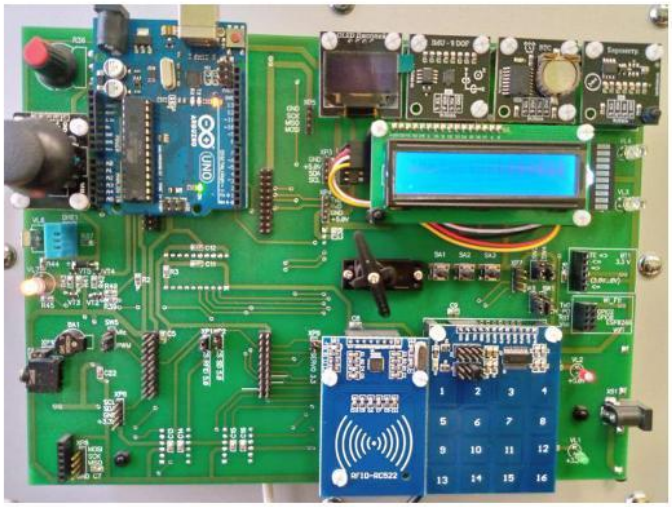


Рисунок 2. Демонстрация «мигания» светодиода.

Программа, реализующая вышеописанные действия со светодиодом:

```
void setup()
{
  pinMode(LED_BUILTIN, OUTPUT);  // установка режима работы порта D13 OUTPUT
}


void loop()
{
  digitalWrite(LED_BUILTIN, HIGH);  // подаём на порт высокий уровень
  delay(1000);                      // задержка 1000мс = 1с
  digitalWrite(LED_BUILTIN, LOW);   // подаём на порт низкий уровень
  delay(1000);                      // задержка 1000мс = 1с
}
```

2. Воспользоваться стандартными функциями для генерации ШИМ:

Номер варианта в журнале меньше 8, поэтому нужно сгенерировать ШИМ на 13 выводе Arduino. Плавно зажигать и гасить светодиод с некоторым периодом, яркостью в пределах от 0 до номера в журнале, умноженного на 30 с шагом, равным номеру в журнале.

Итак, данные по варианту: яркость от 0 до 1 * 30; шаг – 1. В связи с тем, что светодиод начинает гореть от значения равного 128, то выполнение задания невозможно. Поменяем условие, чтобы визуально замечать, что светодиод действительно плавно зажигается и гаситься.

Пусть яркость светодиода меняется от 0 до 255, с шагом в 1, каждые 100мс и, от 255 до 128 (потому что при значении меньшем светодиод уже не светиться) с тем же шагом. Таким образом, мы сможем визуально заметить, что светодиод не только зажжётся, но и ещё будет менять яркость (рис. 3).

Для вышеописанных действий организуем 2 цикла. Первый будет увеличивать яркость светодиода от 0 до 255, а второй уменьшать от 255 до 128. По окончанию выполнения двух циклов, светодиод погаснет, т.к. при переходе на первый произойдёт сброс яркости в нуль.

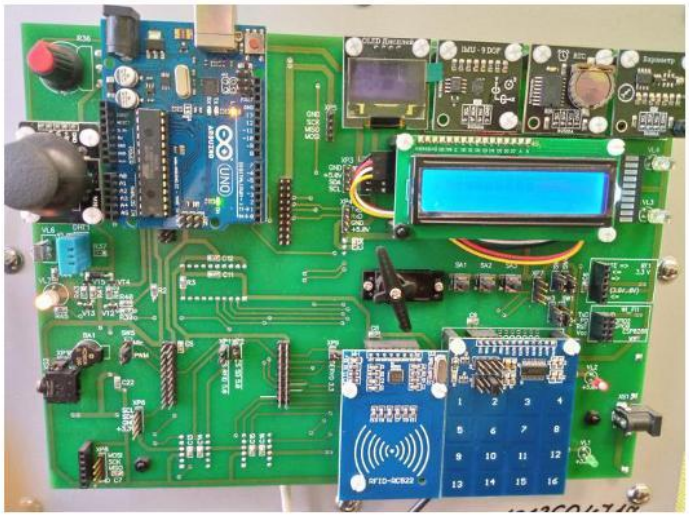


Рисунок 3. Демонстрация изменения яркости светодиода.

Программа, реализующая вышеописанные действия со светодиодом, располагается на следующей странице.

```
void setup()
{
  pinMode(13, OUTPUT);        // установка режима работы порта D13 OUTPUT
}

void loop()
{
  size_t i;                          // инициализация переменной счётчика i

  for (i = 0; i <= 255; i = i + 1) // первый описанный цикл: плавное
зажигание
  {
    analogWrite(13, i);
    delay(100);
  }

  while (i >= 128)                   // второй описанный цикл: плавное гашение
  {
    analogWrite(13, i);
    delay(100);
    i -= 1;
  }
}
```

Смотрите выполнение 3 задания на следующей странице.

3. Получить напряжение на центральном выводе потенциометра при помощи АЦП и вывести его в COM порт. Вращая ручку, убедиться, что напряжение изменяется. С помощью функции map изменить диапазон получаемого значения в диапазон от 0 до номера в журнале. Если диапазон получиться слишком маленьким, умножить максимальное значение на 10.

Для начала просто получим напряжение на центральном выводе потенциометра при помощи АЦП. Для того выполним analogRead(A0) и выведем полученное значение в COM порт (рис. 4):

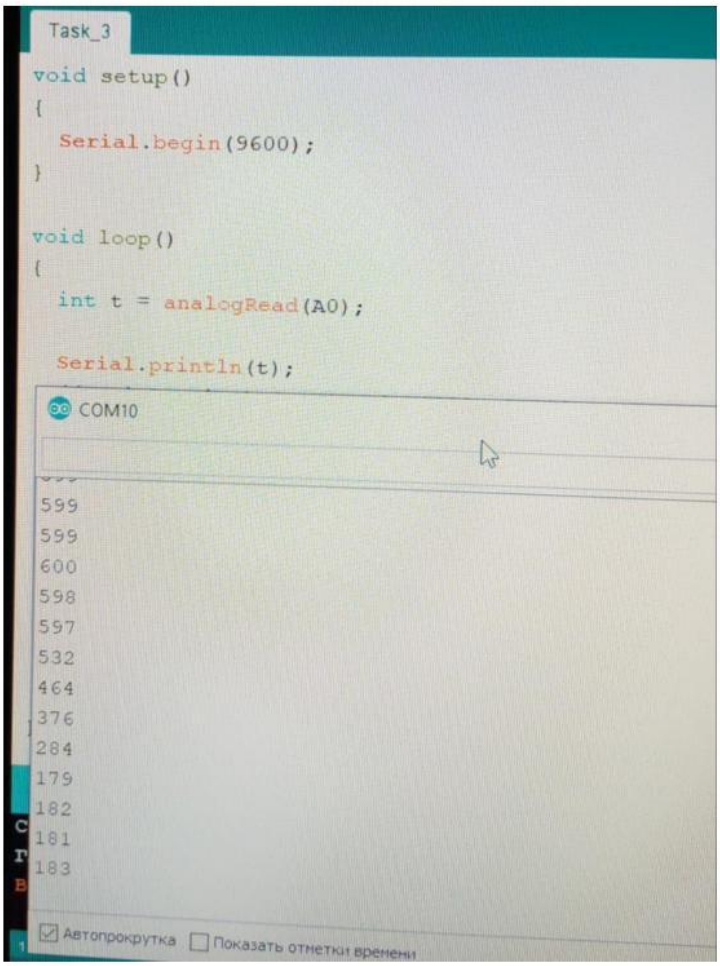


Рисунок 4. Проверка вывода в COM порт значения с A0.

Теперь воспользуемся функцией map. Начальный диапазон будет маленьким, т.к. номер варианта = 1. Следовательно, используем данные выше инструкции и умножим на 10. Выполним те же действия и убедимся, что вывод изменился (см. рис. 5 на след. стр.).

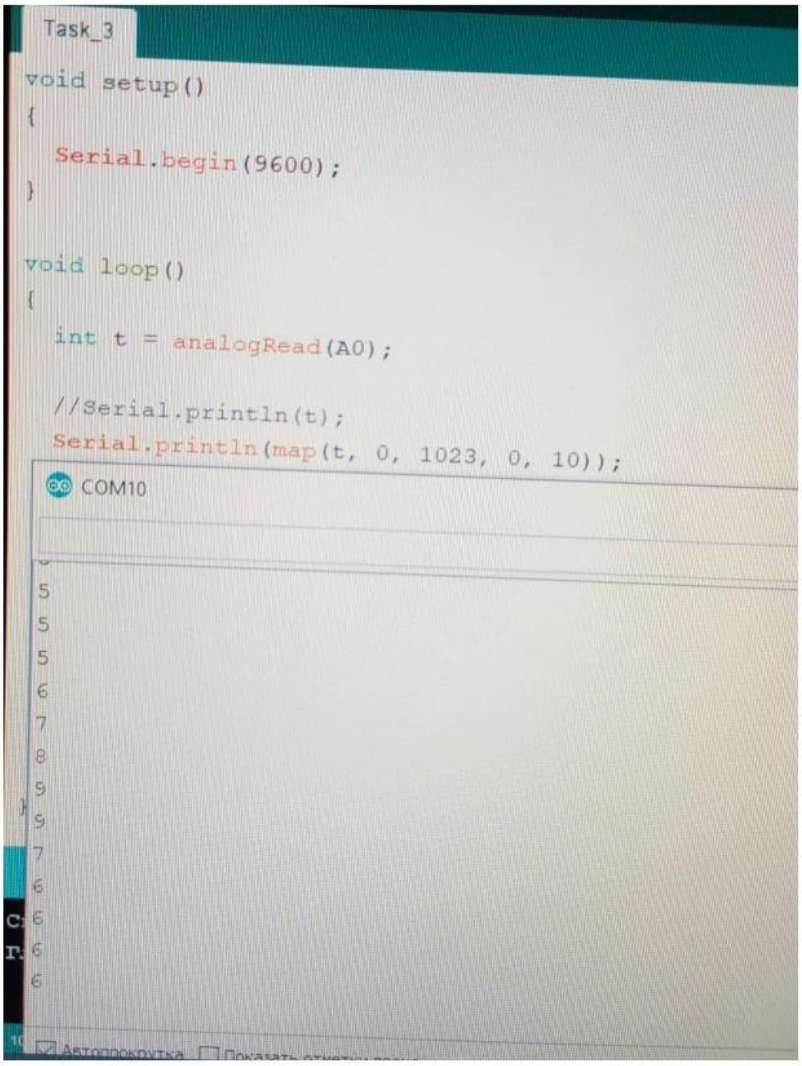


Рисунок 5. Вывод в COM порт обработанного значения с A0.

Таким образом, мы с помощью функции map изменили выводимый в COM порт диапазон измеряемого напряжения. Программа, реализующая вышеописанные действия со светодиодом, расположена ниже:

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  int t = analogRead(A0);

  //Serial.println(t);
  Serial.println(map(t, 0, 1023, 0, 10));
  /*
   *  Функция map переносит значение из текущего диапазона в новый диапазон
значений,
   * заданный параметрами – map(value, fromLow, fromHigh, toLow, toHigh).
```

```
    * value: значение для переноса
    * fromLow: нижняя граница текущего диапазона
    * fromHigh: верхняя граница текущего диапазона
    * toLow: нижняя граница нового диапазона
    * toHigh: верхняя граница нового диапазона
    */
  delay(1000);
}
```

Смотрите выполнение 4 задания на следующей странице.

4. С помощью интерфейса I2C изменить состояние расширителя портов. Зажечь несколько светодиодов на световой шкале, зажечь RGB светодиоды. Полоски на шкале загораются при низком уровне на выводе (схема с общим анодом), а RGB светодиоды при высоком. Сделать задание, соответствующее своему варианту по следующим преобразованиям: номер в журнале % 6 + 1.

Вычисление номера задания. Номер в журнале = 1, следовательно, задание для выполнения будет 1 % 6 + 1 = 1 + 1 = 2.

Формулировка 2-го задания: Зажечь полоски световой шкалы через одну, начиная со второй.

В ходе выполнения данного задания, мы будем осуществлять взаимодействие с расширителем портов, т.е. с микросхемой, по сути, предоставляющей дополнительные GPIO порты (в данном стенде – микросхемы PCA9538). Схема подключения расширителей изображена ниже, на рисунке 6.

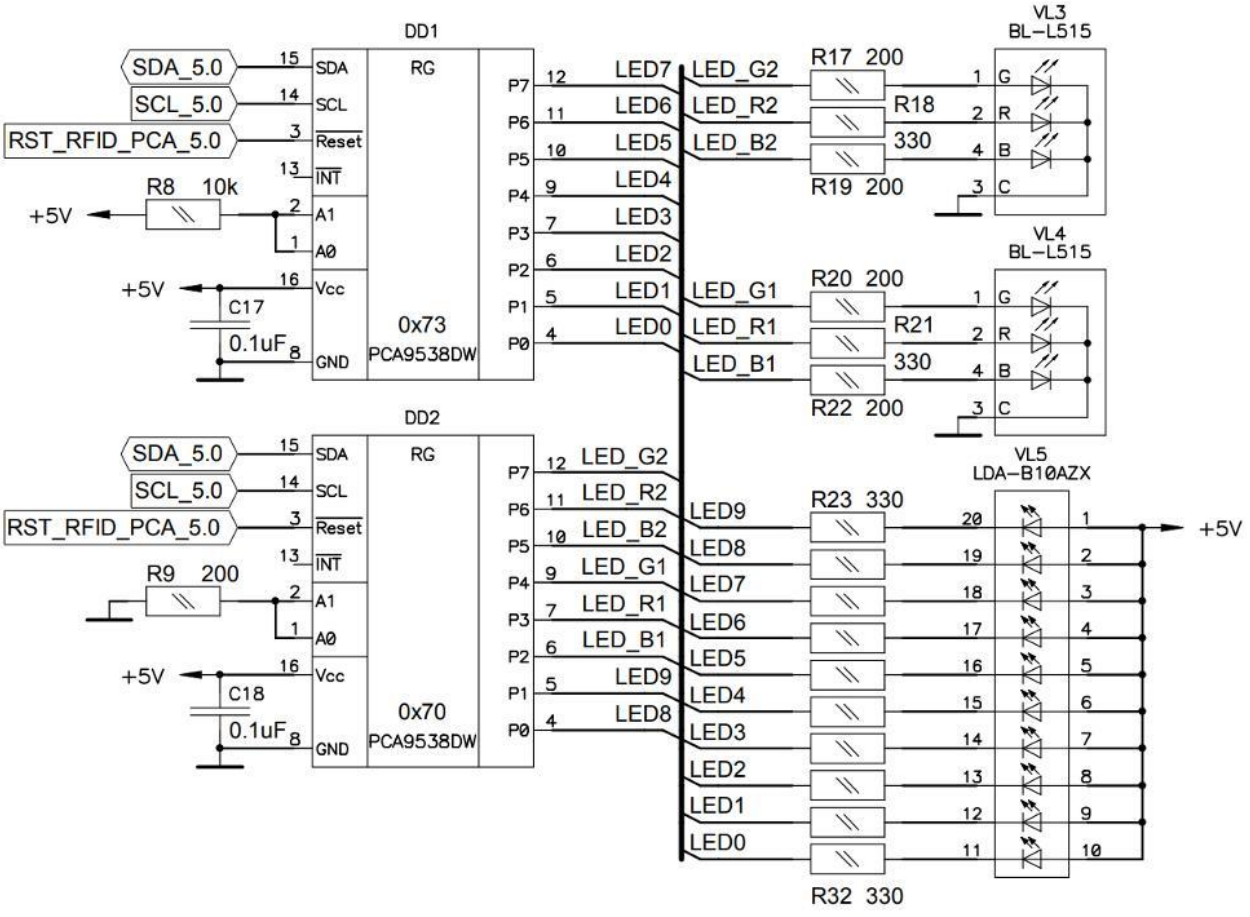


Рисунок 6. Схема подключения расширителей PCA9538.

Глядя на схему подключения расширителей (рис. 6), примем во внимание, что два последних светодиода шкалы подключены к регистру с адресом 0x70. Таким образом, для зажигания всей шкалы, нужно будет работать сразу с двумя регистрами.

Подключение со стороны микроконтроллера показано ниже на рисунке 7.

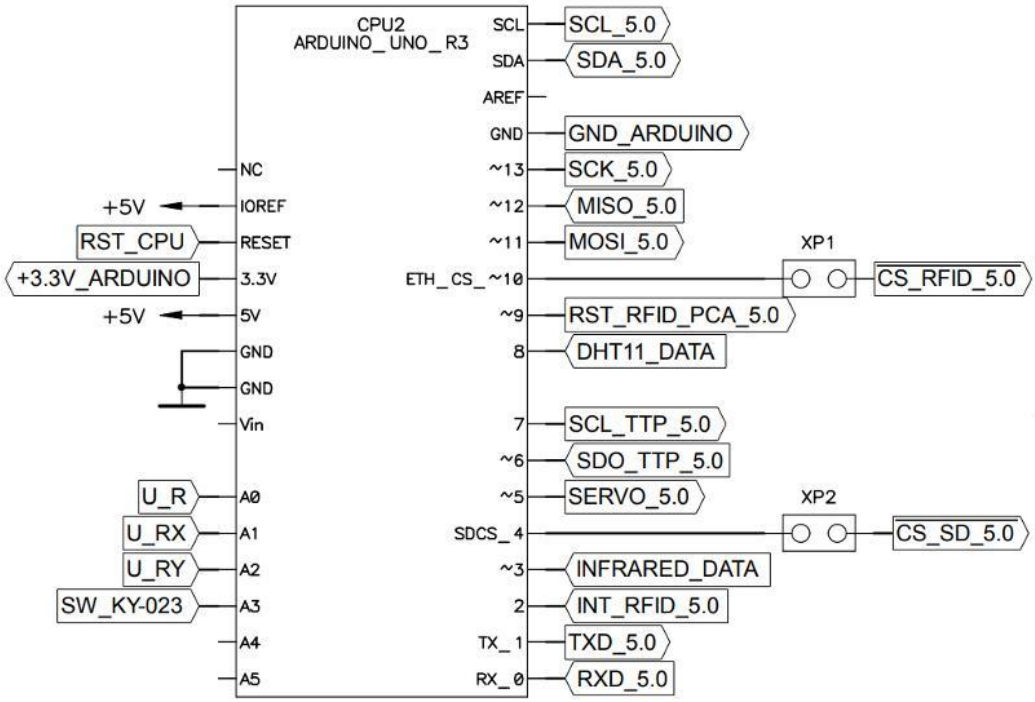


Рисунок 7. Схема подключения платы Arduino Uno.

Изучив схему, начнём реализовывать программу. Вначале сконфигурируем работу только со световой шкалой. RGB-светодиоды в этом задании нам не понадобятся. Т.е. настраиваем все ножки на вывод. Теперь можно зажигать полоски световой шкалы через одну, начиная со второй. Должны засветиться шкалы 2, 4, 6, 8, 10 (см. рис.8).

Чтобы зажечь 4, 6, 8 и 10 шкалы, нужно записать значение 170: 128 + 32 + 8 + 2. Это есть 7, 5, 3 и 1 степени двойки, т.е. будет передан байт 010101010. Загорятся те шкалы, разряды которых нулевые, так как светодиодная шкала загорается при уровне сигнала LOW. Для зажигания 2 шкалы и не зажигания RGB-светодиодов, нужно записать значение 2: 00000010. Программа, реализующая вышеописанные действия со светодиодом, расположена ниже, после рисунка 8.

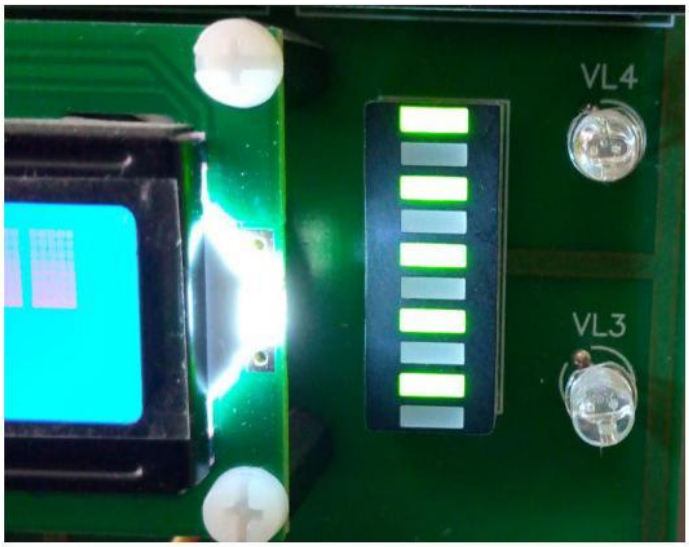


Рисунок 8. Демонстрация работы светодиодной шкалы.

```
#include <Wire.h>

void setup()
{
  Wire.begin(); // join i2c bus

  // Настройка ленты с 9 по 10 индикатор и светодиодов на вывод
  Wire.beginTransmission(0x70); // transmit to device #8
  Wire.write(0x03);
  Wire.write(0x00);
  Wire.endTransmission();       // stop transmitting

  // Настройка ленты с 1 по 8 индикатор на вывод
  Wire.beginTransmission(0x73); // transmit to device #8
  Wire.write(0x03);
  Wire.write(0x00);
  Wire.endTransmission();       // stop transmitting

  delay(500);
}

void loop()
{
  // Зажечь на ленте шкалы 4, 6, 8, 10
  Wire.beginTransmission(0x73); // transmit to device #8
  Wire.write(0x01);
  Wire.write(170);
  Wire.endTransmission();       // stop transmitting
  // Зажечь на ленте шкалцу 2
  Wire.beginTransmission(0x70); // transmit to device #8
  Wire.write(0x01);
  Wire.write(2);
  Wire.endTransmission();       // stop transmitting

  delay(500);

}
```

5. Написать микро-библиотеку (несколько функций), реализующую взаимодействие с расширителем портов PCA9538. Должна быть возможность устанавливать режим любого порта на расширителе, считывать уровень сигнала на нём, устанавливать выходной уровень, включать инверсию по входу. Формировать нужные значения регистров можно при помощи маски/битового поля.

Номер в журнале меньше 8:

Используя полученную библиотеку, реализовать отображение числа при помощи световой шкалы. Задействована должна быть вся шкала, включая 2 нижних полоски.

Сначала должно произойти тестирование шкалы – отображение чисел от 1 до 10, с шагом 1 за 1 секунду. Затем отображать остаток от деления значения, полученного с помощью analogRead на выводе A0(U_R, центральный вывод потенциометра), на номер в журнале. При вращении ручки потенциометра отображаемое число должно меняться.

Напишем микро-библиотеку, где для удобства работы со световой шкалой опишем класс Device. В нём будут все необходимые функции: взаимодействие с расширителем портов PCA9538, установка режима любого порта на расширителе, считывание уровня сигнала на нём, установка выходного уровня, включение инверсии по входу и другие. Написанный класс изображён в коде ниже:

```
#ifndef deviceLib_h
#define deviceLib_h

#include <Wire.h>

class Device  // Класс устройства. Через него будем управлять выбираемым
устройством.
{
public:
        Device(byte adress);                        // Конструктор
        void setAdress(byte adress);                // Метод, устанавливающий
адрес используемого устройства
        void setCommandByte(byte comandByte);       // Метод, устанавливающий
байт-команду (режим порта)
        void setVariableByte(byte variableByte); // Метод, устанавливающий
байт-значение
        void sendAll();                                                    //
Метод, осуществляющий передачу адреса, байта команды и байта значений по I2C
        uint8_t readLevelSignal();                                  // Метод,
```

```
считывающий уровень сигнала порта
        void setOutputLevel(byte value);                // Метод,
устанавливающий выходной уровень
        void activateInversionInBits(byte bytes);// Метод, включающий
инверсию по входу

private:
        uint8_t _adress;          // Переменная, содержащая адрес устройства
        uint8_t _commandByte;     // Переменная, содержащая байт-команду,
выбирающая режим обращения к устройству (ввод, вывод, конфигурирование и
т.п.)
        uint8_t _variableByte;    // Переменная, содержащая байт-значение

};


// Реализация методов
Device::Device(byte adress)
{
        _adress = adress;
}

void Device::setAdress(byte adress) {_adress = adress;}

void Device::setCommandByte(byte comandByte) {_commandByte = comandByte;}

void Device::setVariableByte(byte variableByte) {_variableByte =
variableByte;}

void Device::sendAll()
{
        Wire.beginTransmission(_adress); // transmit to device
        Wire.write(_commandByte);
        Wire.write(_variableByte);
        Wire.endTransmission();          // stop transmitting
}

uint8_t Device::readLevelSignal()
{
        Wire.beginTransmission(_adress); // transmit to device
        Wire.write(0x00);
        _variableByte = Wire.read();
        Wire.endTransmission();          // stop transmitting
}

void Device::setOutputLevel(byte value)
{
        Wire.beginTransmission(_adress); // transmit to device
        Wire.write(0x01);
        _variableByte = value;
        Wire.write(_variableByte);
        Wire.endTransmission();          // stop transmitting
}
```

```
void Device::activateInversionInBits(byte bytes)
{
        Wire.beginTransmission(_adress); // transmit to device
        Wire.write(0x02);
        Wire.write(bytes);
        Wire.endTransmission();          // stop transmitting
}

#endif
```

Используя нашу написанную библиотеку, создадим 2 объекта: lenta_1_8 и lenta_9_10AndDiodes. Вначале конфигурируем наши созданные объекты: настраиваем ножки на вывод (0x03 – конфигурация) и обнуляем все световые шкалы. После подготовительных мероприятий, проводим тестирование шкалы. Сказано, что нужно отобразить числа от 1 до 10, с шагом 1 за 1 секунду. Не сказано, как нужно интерпретировать числа для световой шкалы.

Пусть каждое число отвечает за свою шкалу, т.е. отображению 1 соответствует первая сверху шкала (рис. 8), отображению 2 соответствует вторая сверху шкала и т.д. Тогда, при тестировании световой шкалы, будем зажигать каждую шкалу сверху вниз поочерёдно, с длительностью свечения 1 секунду, тем самым «эмулируя» отображение чисел (рис. 9). После окончания теста, выключим/погасим все шкалы.

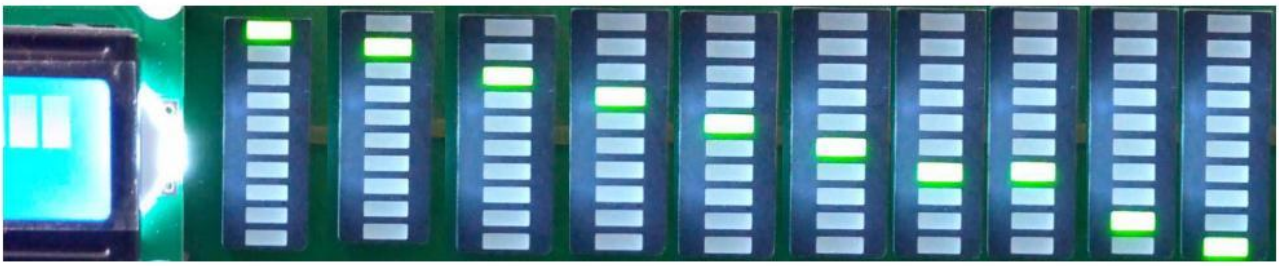


Рисунок 9. Демонстрация работы светодиодной шкалы.

Теперь можно отображать остаток от деления значения, полученного с помощью analogRead на выводе A0 (U_R, центральный вывод потенциометра), на номер в журнале. Т.к. номер в журнале = 1, то ни одна световая шкала не будет зажжена, потому что при делении на 1 остаток всегда равен нулю. Для выполнения задания вновь изменим его условия. Пусть мы будем отображать остаток от деления на t = 10 (можно поставить любое другое значение). Также для удобства видимости результата воспользуемся изменением диапазона

измеряемых значений с вывода A0 от 0 до 10 (также возможно изменить этот диапазон или вовсе его убрать, т.е. использовать стандартный). Вращая ручку можно наблюдать, как световая шкала переключается с одной полоски на другую (серия рисунков 10 – 16). Программа, реализующая вышеописанные действия, расположена после рисунков 10 – 16.

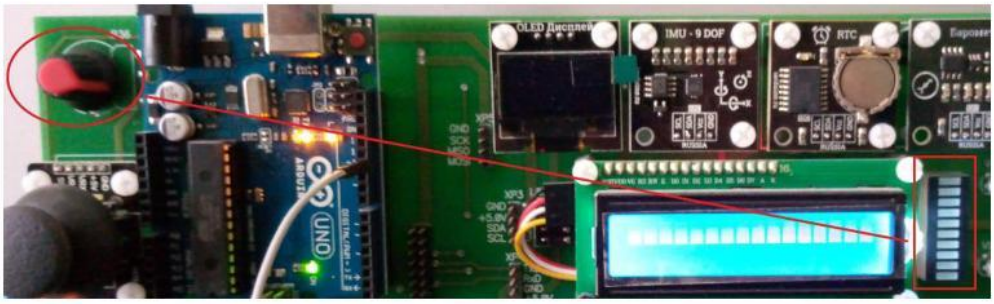


Рисунок 10. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 1).

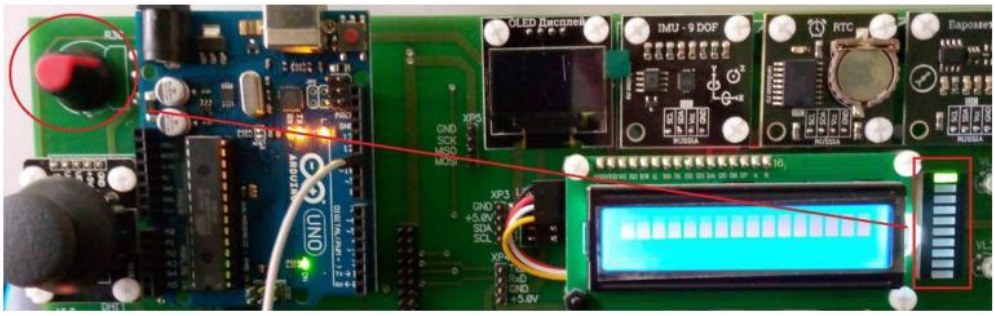


Рисунок 11. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 2).

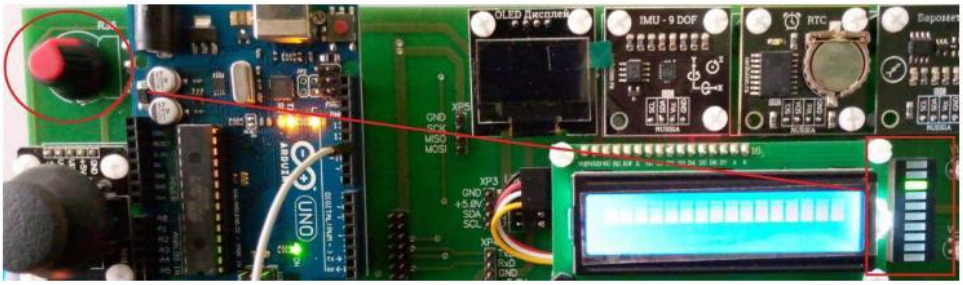


Рисунок 12. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 3).

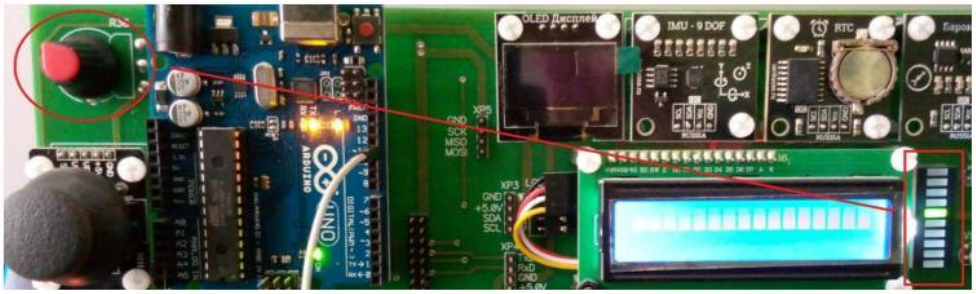


Рисунок 13. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 4).

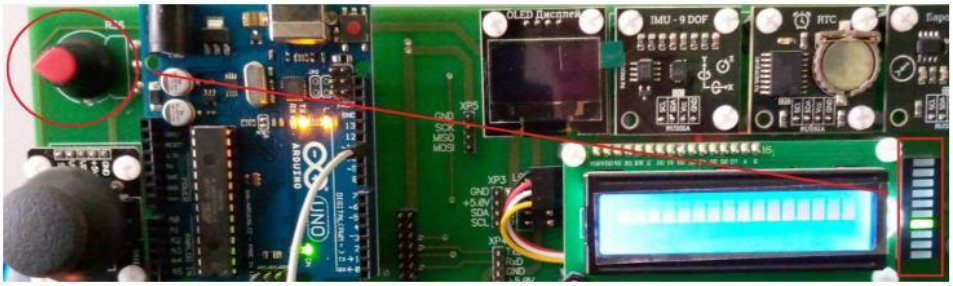


Рисунок 14. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 5).

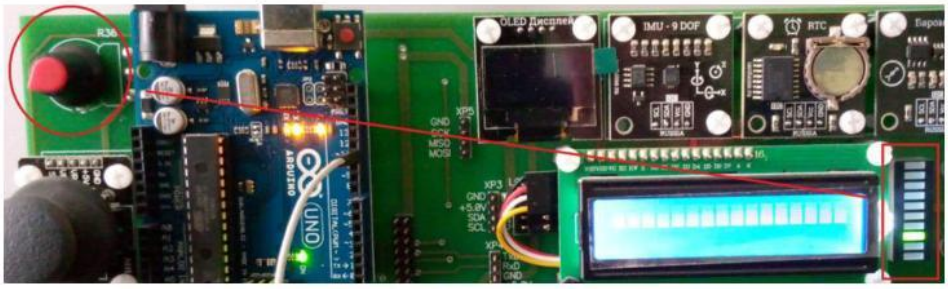


Рисунок 15. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 6).

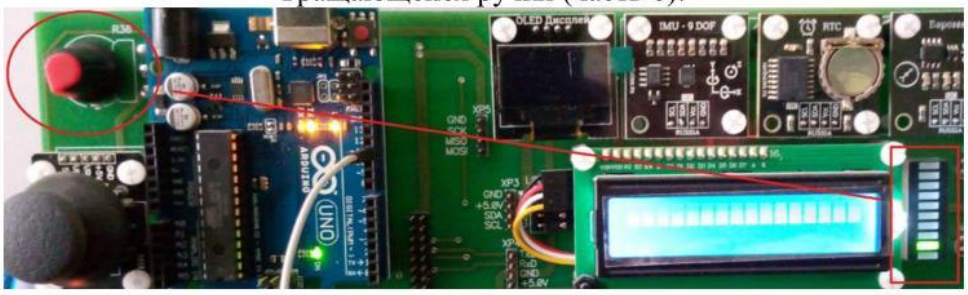


Рисунок 16. Демонстрация работы светодиодной шкалы в зависимости от положения вращающейся ручки (часть 7).

```
#include "DeviceLib.h"


Device lenta_1_8(0x73);                  // создаём объект lenta_1_8 и
указываем адрес устройства
Device lenta_9_10AndDiodes(0x70);        // создаём объект lenta_9_10AndDiodes
и указываем адрес устройства

void setup()
{
  // конфигурируем все устройства
  Wire.begin();                          // join i2c bus

  // Настройка световой шкалы со шкалами 1 - 8 на приём
  lenta_1_8.setCommandByte(0x03);
  lenta_1_8.setVariableByte(0x00);
  lenta_1_8.sendAll();
```

```
  // Настройка световой шкалы со шкалами 9 - 10 на приём
  lenta_9_10AndDiodes.setCommandByte(0x03);
  lenta_9_10AndDiodes.setVariableByte(0x00);
  lenta_9_10AndDiodes.sendAll();

  // Обнуление световой шкалы
  lenta_1_8.setCommandByte(0x01);
  lenta_1_8.setVariableByte(255);
  lenta_1_8.sendAll();
  lenta_9_10AndDiodes.setCommandByte(0x01);
  lenta_9_10AndDiodes.setVariableByte(3);
  lenta_9_10AndDiodes.sendAll();

  // Тестирование шкалы: отображаем числа от 1 до 10. Не сказано, как
отображать, поэтому 1 деление - одна цифра. 1-е деление - один, 2-е деление
- два и т.д.
  for (int i = 0; i <= 7; i += 1)
  {
    lenta_1_8.setCommandByte(0x01);
    lenta_1_8.setVariableByte(~(1<<i));
    lenta_1_8.sendAll();
    delay(1000);
  }
  lenta_1_8.setVariableByte(255);
  lenta_1_8.sendAll();

  lenta_9_10AndDiodes.setVariableByte(2);
  lenta_9_10AndDiodes.sendAll();
  delay(1000);
  lenta_9_10AndDiodes.setVariableByte(1);
  lenta_9_10AndDiodes.sendAll();
  delay(1000);
  lenta_9_10AndDiodes.setVariableByte(3);
  lenta_9_10AndDiodes.sendAll();

}

void loop()                    // реализация отображения остатка от деления
значения с порта A0 на световой шкале
{
  int t = analogRead(A0);
  t = map(t, 0, 1023, 0, 10);
  t %= 10;

  if (t == 0)
  {
    lenta_9_10AndDiodes.setCommandByte(0x01);
    lenta_9_10AndDiodes.setVariableByte(3);
    lenta_9_10AndDiodes.sendAll();
    lenta_1_8.setCommandByte(0x01);
    lenta_1_8.setVariableByte(255);
    lenta_1_8.sendAll();
  }
  else
  {
    if (t <= 8)
    {
```

```
      lenta_9_10AndDiodes.setCommandByte(0x01);
      lenta_9_10AndDiodes.setVariableByte(3);
      lenta_9_10AndDiodes.sendAll();
      lenta_1_8.setCommandByte(0x01);
      lenta_1_8.setVariableByte(~(1<<(t-1)));
      lenta_1_8.sendAll();
    }
    else
    {
      t -= 8;
      lenta_9_10AndDiodes.setCommandByte(0x01);
      if (t == 1)
      {
        lenta_1_8.setCommandByte(0x01);
        lenta_1_8.setVariableByte(255);
        lenta_1_8.sendAll();
        lenta_9_10AndDiodes.setVariableByte(2);
        lenta_9_10AndDiodes.sendAll();
      }
      else
      {
        lenta_1_8.setCommandByte(0x01);
        lenta_1_8.setVariableByte(255);
        lenta_1_8.sendAll();
        lenta_9_10AndDiodes.setVariableByte(1);
        lenta_9_10AndDiodes.sendAll();
      }
    }
  }
  delay(1000);
}
```

Вывод: в ходе выполнения лабораторной работы мы изучили среду разработки ARDUINO IDE и познакомились со стандартными библиотеками для работы с портами и интерфейсами, исследовали некоторую часть возможностей микроконтроллера Atmega 328p, научились работать с GPIO портами, с расширителем портов, с помощью интерфейса I2C взаимодействовать с подключенными устройствами. Научились программировать исследуемый микроконтроллер, программировать вешние устройства, подключенные к нему и использовать их по своему назначению. Также применили навыки ООП, реализовав свою микро-библиотеку для взаимодействия с устройствами с помощью интерфейса I2C.